

**National University of Computer & Emerging
Sciences**
Karachi Campus



RetroCalc
Old-School Calculator

Project Documentation
Compiler Construction
Section: 6J

Group Members:
23k-2001 Muzammil Siddiqui
23k-0532 Rayyan Ali
23k-0568 Azaan Hussain

Language Reference Manual

1.1 Overview

RetroCalc is a simple, interpreted programming language designed for arithmetic computation, variable storage, and basic program flow control. Its syntax is intentionally minimal, making it an ideal teaching language for demonstrating the core phases of a compiler: lexical analysis, parsing, semantic analysis, intermediate code generation, optimization, and interpretation.

1.2 Lexical Conventions

1.2.1 Character Set

RetroCalc source files use ASCII characters. Whitespace (spaces and tabs) is ignored except as a token separator. Newlines act as statement terminators.

1.2.2 Comments

A comment begins with the hash character (#) and extends to the end of the line. Comments produce no tokens.

```
# This is a comment
let x = 5    # inline comment
```

1.2.3 Keywords

The following identifiers are reserved and may not be used as variable names:

```
let    print    if    loop    end
```

1.2.4 Token Types

Token Type	Pattern / Value	Example
INT_LITERAL	One or more decimal digits	42, 0, 100
FLOAT_LITERAL	Digits, '.', digits	3.14, 2.5, 0.0
IDENTIFIER	[a-zA-Z_][a-zA-Z0-9_]*	x, total, my_var
PLUS	+	a + b
MINUS	-	a - b
STAR	*	a * b
SLASH	/	a / b
MOD	%	a % b
ASSIGN	=	let x = 5
EQ	==	x == 5
NEQ	!=	x != 5

GT	>	x > 3
LT	<	x < 3
GTE	>=	x >= 3
LTE	<=	x <= 3
LPAREN	(	(a + b)
RPAREN	)	(a + b)
NEWLINE	\n – statement separator	(end of line)
EOF	End of source	(end of file)

1.3 Data Types

RetroCalc supports two primitive types:

- **int** — a whole number (Python int internally). Results from integer literals and integer-only arithmetic.
- **float** — a floating-point number (Python float internally). Results from float literals or any expression mixing int and float operands.

Type promotion rule: if either operand in a binary expression is float, the result is float.

1.4 Grammar

The complete formal grammar in BNF notation:

```

program      ::= statement*
statement    ::= let_stmt | print_stmt | if_stmt | loop_stmt
let_stmt     ::= 'let' IDENT '=' expression NEWLINE
print_stmt   ::= 'print' expression NEWLINE
if_stmt      ::= 'if' expression NEWLINE statement* 'end' NEWLINE
loop_stmt    ::= 'loop' expression NEWLINE statement* 'end' NEWLINE
expression   ::= comparison
comparison   ::= addition (('>'| '<'| '>='| '<='| '=='| '!=') addition)?
addition     ::= multiplication (('+'| '-') multiplication)*
multiplication ::= unary (('*'| '/'| '%') unary)*
unary        ::= '-' unary | primary
primary      ::= INT_LITERAL | FLOAT_LITERAL | IDENT | '(' expression ')'
```

1.5 Statements

1.5.1 Variable Declaration — let

Declares a variable and assigns it the value of an expression. All variables must be declared before use.

```

let <identifier> = <expression>

let x = 10
let result = x * 2 + 1
```

1.5.2 Print Statement — print

Evaluates an expression and prints the result to standard output.

```
print <expression>

print x
print 3 + 4 * 2
```

1.5.3 Conditional — if / end

Evaluates a condition. If the condition is non-zero (truthy), the body statements are executed. The block is terminated by end. There is no else clause.

```
if <expression>
  <statements>
end

if x > 5
  print x
end
```

1.5.4 Loop — loop / end

Repeats the body statements a fixed number of times. The count expression is evaluated once before the first iteration.

```
loop <expression>
  <statements>
end

loop 3
  print x
end
```

1.6 Expressions and Operator Precedence

Operators are listed from lowest to highest precedence:

Level 1 (lowest)	Comparison: > < >= <= == !=
Level 2	Addition / Subtraction: + -
Level 3	Multiplication / Division / Modulo: * / %
Level 4 (highest)	Unary negation: -

All arithmetic operators are left-associative. Parentheses can override precedence.

```
let a = 2 + 3 * 4      # result = 14 (not 20)
let b = (2 + 3) * 4    # result = 20
let c = -a             # result = -14
```

1.7 Scoping Rules

RetroCalc uses a flat, single global scope. All variables declared anywhere in the program share the same namespace. There is no block-scoping — variables declared inside if or loop blocks

remain accessible after the block ends. Re-assigning a variable with `let` overwrites its previous value.

1.8 Error Handling

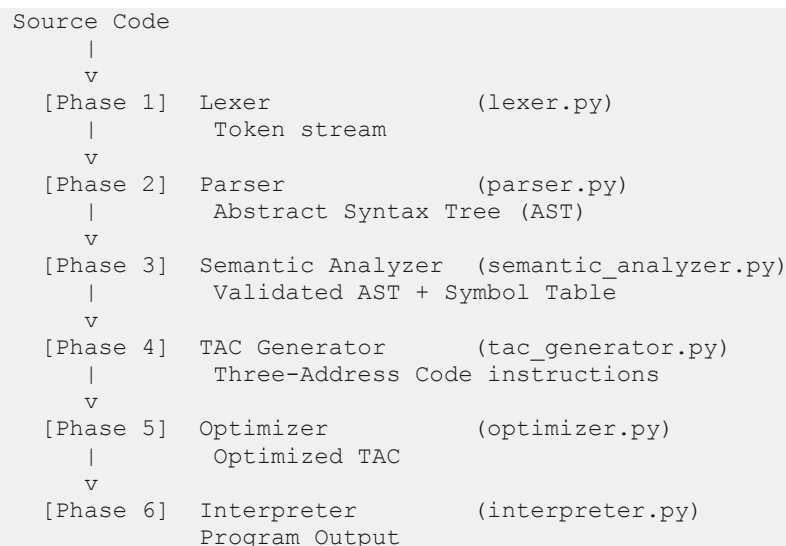
The compiler raises three categories of errors:

- `LexerError` — raised when an unexpected character is encountered during tokenization. Reports the line and column number.
- `ParserError` — raised when the token stream does not match the grammar. Reports the unexpected token, its value, and position.
- `SemanticError` — raised when a variable is used before it has been declared. Additionally, warnings are issued for variables that are declared but never read.

Compiler Architecture

2.1 Pipeline Overview

The RetroCalc compiler is structured as a classical multi-phase pipeline. Source text flows through each phase sequentially, being transformed at each step until the program is executed.



2.2 Module Descriptions

Phase 1 — Lexical Analyzer (lexer.py)

The `Lexer` class scans the source string character by character, implementing a hand-written Deterministic Finite Automaton (DFA). It produces a flat list of `Token` objects, each carrying a type, value, line number, and column number.

Key implementation details:

- `_advance()` consumes one character and updates line/column tracking.

- `_peek()` looks one character ahead without consuming, enabling two-character operator recognition (`>=`, `<=`, `==`, `!=`).
- `_read_number()` handles both integers and floats, raising `LexerError` on a second decimal point.
- `_read_identifier_or_keyword()` reads an alphanumeric sequence and checks it against the `KEYWORDS` dictionary.
- Newlines are emitted as `NEWLINE` tokens (deduplicated) to serve as statement separators.
- Comment lines (starting with `#`) are silently consumed with `_skip_comment()`.

Phase 2 — Parser (`parser.py`)

The Parser implements a recursive-descent parsing strategy. Each non-terminal in the grammar corresponds to a private method. The parser consumes the token stream produced by the Lexer and constructs an Abstract Syntax Tree (AST) of typed node objects.

AST node types (defined in `ast_nodes.py`):

- `Program` — root node containing a list of statements.
- `LetStatement` — stores variable name and expression node.
- `PrintStatement` — stores the expression to print.
- `IfStatement` — stores the condition expression and body statement list.
- `LoopStatement` — stores the count expression and body statement list.
- `BinaryOp` — stores operator string, left node, and right node.
- `UnaryOp` — stores operator and operand node.
- `NumberLiteral` — stores the literal int or float value.
- `Identifier` — stores the variable name string.

Operator precedence is enforced by the call chain: `_parse_comparison` calls `_parse_addition`, which calls `_parse_multiplication`, which calls `_parse_unary`, which calls `_parse_primary`. This ensures, for example, that `*` binds more tightly than `+`.

Phase 3 — Semantic Analyzer (`semantic_analyzer.py`)

The `SemanticAnalyzer` performs a single-pass walk of the AST, validating the program's semantic correctness and building a symbol table.

Checks performed:

- Declaration-before-use: every `Identifier` reference is checked against the `declared_vars` dictionary. An undefined variable raises `SemanticError` immediately.
- Type inference: types (int or float) are inferred bottom-up. `NumberLiteral` nodes carry their type; `BinaryOp` nodes propagate float if either operand is float.
- Unused variable warnings: after the full walk, any variable in `declared_vars` that does not appear in `used_vars` generates a warning message.

Phase 4 — TAC Generator (`tac_generator.py`)

The TACGenerator walks the validated AST and emits a flat list of TACInstruction objects in Three-Address Code format. Each instruction has at most one operator, two operand slots (`arg1`, `arg2`), and one result slot.

Instruction types emitted:

- **ASSIGN** — copies a value: `result = arg1`
- **Binary ops** (`+`, `-`, `*`, `/`, `%`, `>`, `<`, `>=`, `<=`, `==`, `!=`) — `result = arg1 op arg2`
- **UNARY_MINUS** — `result = -arg1`
- **PRINT** — outputs `arg1`
- **LABEL** — marks a jump target
- **GOTO** — unconditional jump to a label
- **IF_FALSE** — conditional jump: if `arg1` is zero, jump to label `arg2`

Temporary variables are named `t0`, `t1`, `t2`, ... and are allocated with `_new_temp()`. Labels are named `L0`, `L1`, `L2`, ... and allocated with `_new_label()`.

Phase 5 — Optimizer (`optimizer.py`)

The Optimizer applies three sequential optimization passes to the TAC instruction list:

- **Constant Folding** — evaluates binary operations where both operands are known compile-time constants. Example: `t0 = 3 + 5` becomes `t0 = 8`.
- **Constant Propagation** — tracks which temporaries and variables hold constant values and substitutes those constants into subsequent instructions, enabling further folding opportunities.
- **Dead Code Elimination** — scans the entire instruction list for temporaries (`t0`, `t1`, ...) that are assigned but never read, and removes those assignment instructions. User-declared variables are never eliminated.

The optimizer records a human-readable log of every change it makes, accessible via `get_report()`.

Phase 6 — Interpreter (`interpreter.py`)

The Interpreter executes the optimized TAC instruction list directly. It maintains a memory dictionary mapping variable and temporary names to their current values.

Execution rules:

- **ASSIGN** — writes `arg1` to `memory[result]`.
- **Binary ops** — reads operands from memory or uses literal values, computes the result, writes to `memory[result]`.
- **PRINT** — resolves the value from memory and prints it.
- **LABEL** — a no-op during execution (used as jump targets).
- **GOTO** — sets the program counter to the instruction following the named label.
- **IF_FALSE** — checks the condition value; if zero, jumps to the named label.

2.3 File Structure

main.py	Entry point: CLI argument parsing, orchestrates all phases
tokens.py	TokenType enum and Token dataclass
lexer.py	Lexer class — character scanner / DFA
ast_nodes.py	AST node dataclasses + pretty printer
parser.py	Parser class — recursive-descent
semantic_analyzer.py	SemanticAnalyzer class — symbol table + type inference
tac_generator.py	TACGenerator class + TACInstruction dataclass
optimizer.py	Optimizer class — 3 optimization passes
interpreter.py	Interpreter class — TAC executor
errors.py	LexerError, ParserError, SemanticError custom exceptions

2.4 Running the Compiler

```
python main.py <source_file.rc>

# With verbose output (shows tokens, AST, TAC, optimizations):
python main.py <source_file.rc> --verbose
```

Test Suite

The following five test programs cover all major language features. Each program is presented with the source code and the expected output when run through the RetroCalc compiler.

Test 1 — Basic Arithmetic and Variable Assignment

```
# Test 1: arithmetic.rc
let a = 10
let b = 3
let sum  = a + b
let diff = a - b
let prod = a * b
let quot = a / b
let rem  = a % b
print sum
print diff
print prod
print quot
print rem
```

Expected Output:

```
13
7
30
3
1
```

This test verifies that all five arithmetic operators work correctly and that variable declarations are stored and retrieved properly.

Test 2 — Float Arithmetic and Type Promotion

```
# Test 2: floats.rc
let x = 7
let y = 2.0
let result = x / y
let pi_approx = 355.0 / 113
print result
print pi_approx
```

Expected Output:

```
3.5
3.1415929203539825
```

This test verifies that mixing int and float operands promotes the result to float, and that float division is handled correctly.

Test 3 — Conditional Statement (if / end)

```
# Test 3: conditional.rc
let score = 85
let passing = 50
if score > passing
  let grade = 1
  print grade
end
if score < passing
  print 0
end
print score
```

Expected Output:

```
1
85
```

The first if block fires because $85 > 50$ is true. The second if block is skipped because $85 < 50$ is false. The final print executes unconditionally.

Test 4 — Loop Statement with Counter

```
# Test 4: loop.rc
let count = 1
loop 5
  print count
  let count = count + 1
end
```

Expected Output:

```
1
2
3
4
5
```

This test verifies the loop construct executes its body exactly N times (5 here), and that the counter variable updates correctly across iterations.

Test 5 — Operator Precedence, Unary Negation, and Parentheses

```
# Test 5: precedence.rc
let a = 2 + 3 * 4
let b = (2 + 3) * 4
let c = -a + b
let d = 10 % 3
print a
print b
print c
print d
if a != b
    print 1
end
if a == 14
    print 999
end
```

Expected Output:

```
14
20
6
1
1
999
```

This test verifies that: (1) `*` binds before `+`, giving `a=14` not 20; (2) parentheses override precedence; (3) unary minus works; (4) the `%` modulo operator works; (5) `!=` and `==` comparisons both function correctly.

Team Reflection

4.1 Challenges Faced and Solutions

Challenge 1: Two-Character Operator Recognition

One of the first difficulties we encountered in the lexer was correctly handling two-character operators such as `>=`, `<=`, `==`, and `!=`. The naive approach of reading one character at a time would have consumed the first character (e.g., `>`) and emitted it as a single-character GT token before realizing a second character (`=`) followed.

Solution: We implemented a `_peek()` method that reads the next character without advancing the position pointer. Before emitting any single-character operator, the lexer checks whether the following character is `=` and, if so, consumes both characters and emits the two-character token instead. This lookahead approach cleanly solved the ambiguity.

Challenge 2: Enforcing Operator Precedence in the Parser

Getting multiplication to bind more tightly than addition, and comparison operators to bind least tightly of all, required careful structuring of the recursive-descent parser. Initially, we tried to

handle all operators in a single parsing method, which caused expressions like $2 + 3 * 4$ to evaluate left-to-right as 20 instead of the correct 14.

Solution: We restructured the parser into a strict precedence hierarchy: `_parse_comparison` calls `_parse_addition`, which calls `_parse_multiplication`, which calls `_parse_unary`, which calls `_parse_primary`. Each level can only call downward, never upward. This guarantees that operators at lower levels in the call chain bind looser, exactly matching standard arithmetic precedence.

Challenge 3: Implementing Loop TAC with a Counter

The loop statement in RetroCalc takes an expression for the iteration count rather than a boolean condition. Generating correct Three-Address Code for this required initializing an implicit counter temporary, emitting a start label, computing a comparison against the limit on each iteration, and emitting an increment and backward GOTO.

Solution: The TACGenerator's `_gen_loop()` method allocates two fresh temporaries: one for the counter and one for the comparison result. It emits eight instruction groups: limit evaluation, counter init, loop start label, counter < limit comparison, IF_FALSE to end label, body code, counter increment, GOTO start, and end label. Tracing through example programs by hand confirmed the correctness of this pattern.

Challenge 4: Constant Propagation Across Control Flow

The Optimizer's constant propagation pass initially produced incorrect results when it propagated constant values across label boundaries. A variable that held a constant value before an if block might have a different value if the block executed, but the optimizer was still substituting the pre-block constant value in instructions that followed.

Solution: We added a rule that clears the constants dictionary whenever a LABEL instruction is encountered, since labels represent potential control flow merge points where previously tracked constant values can no longer be relied upon. This conservative approach avoids incorrect optimizations at the cost of some missed opportunities, which is the correct trade-off for correctness.

4.2 What We Learned About Compiler Design

Building RetroCalc from scratch gave us hands-on insight into several concepts that had previously felt abstract in lectures:

- Separation of concerns: Dividing the compiler into six distinct phases, each with a clear input and output contract, made each phase independently understandable, testable, and debuggable. A bug in the semantic analyzer did not affect the lexer, and vice versa.
- The power of recursion in parsing: Recursive descent is elegant because the grammar directly shapes the code. Reading the grammar rule for addition and then writing `_parse_addition()` felt natural. We developed an intuition for why left-recursion breaks naive recursive descent.
- Intermediate representations matter: Three-Address Code provided a flat, easy-to-analyze representation of the program. Optimizations that would have been complex to apply to the AST became straightforward scans over a list of instructions.

- Type systems are conservative by design: We observed that our type inference always promotes to float when in doubt, never the other way. This conservatism prevents silent precision loss, at the cost of occasionally producing float results where int would have sufficed.
- Testing must cover edge cases: Our test programs revealed bugs that never appeared in simple happy-path examples, such as empty loop bodies, comparisons in loop counts, and unary negation nested inside binary expressions.

4.3 Future Improvements

Given more time, we would extend RetroCalc in the following directions:

- Else clause for if statements: Currently there is no else or elif. Adding it would require the parser to check for an else keyword after the body and before end, and the TAC generator would need to emit an additional GOTO to skip the else block.
- User-defined functions: Adding a func / return construct would be the most significant language extension. It would require a proper call stack in the interpreter, function symbol table entries in the semantic analyzer, and CALL/RETURN TAC instructions.
- String literals: RetroCalc currently handles numbers only. Adding quoted string literals would require a new STRING_LITERAL token type, a string data type in the semantic analyzer, and print support for string values.
- While loops: The current loop construct runs a fixed number of times. A while condition loop would allow programs to terminate based on runtime conditions, significantly increasing expressive power.
- Better error recovery: Currently the compiler stops at the first error. A production compiler would continue parsing after an error to report multiple errors in a single run, saving the programmer time.
- Register allocation and code generation: Currently the compiler targets a custom interpreter rather than a real machine. Adding a simple code generator targeting MIPS or x86 assembly would complete the full compilation pipeline.

4.4 Individual Contributions

The project was distributed as follows:

23k-2001 Muzammil Siddiqui	Led the implementation of the Lexer (lexer.py) and Token definitions (tokens.py). Designed the DFA for token recognition, implemented two-character operator lookahead, handled comment skipping and newline deduplication. Also contributed to testing and wrote Test Cases 1 and 2.
23k-0532 Rayyan Ali	Implemented the Parser (parser.py) and AST node definitions (ast_nodes.py). Designed the recursive-descent call hierarchy for correct operator precedence, implemented all statement parsers (let, print, if, loop), and implemented the pretty-printer for AST debugging. Wrote Test Cases 3 and 4.
23k-0568 Azaan Hussain	Implemented the Semantic Analyzer (semantic_analyzer.py), TAC Generator (tac_generator.py), Optimizer (optimizer.py), and Interpreter (interpreter.py). Designed the symbol table and type inference rules,

	implemented the three optimization passes, and built the TAC execution engine. Wrote Test Case 5 and integrated all phases in main.py.
--	--

All three members participated in code reviews, joint debugging sessions, and writing this documentation. The project was managed collaboratively with tasks tracked per phase. Each member peer-reviewed the work of the others before integration.